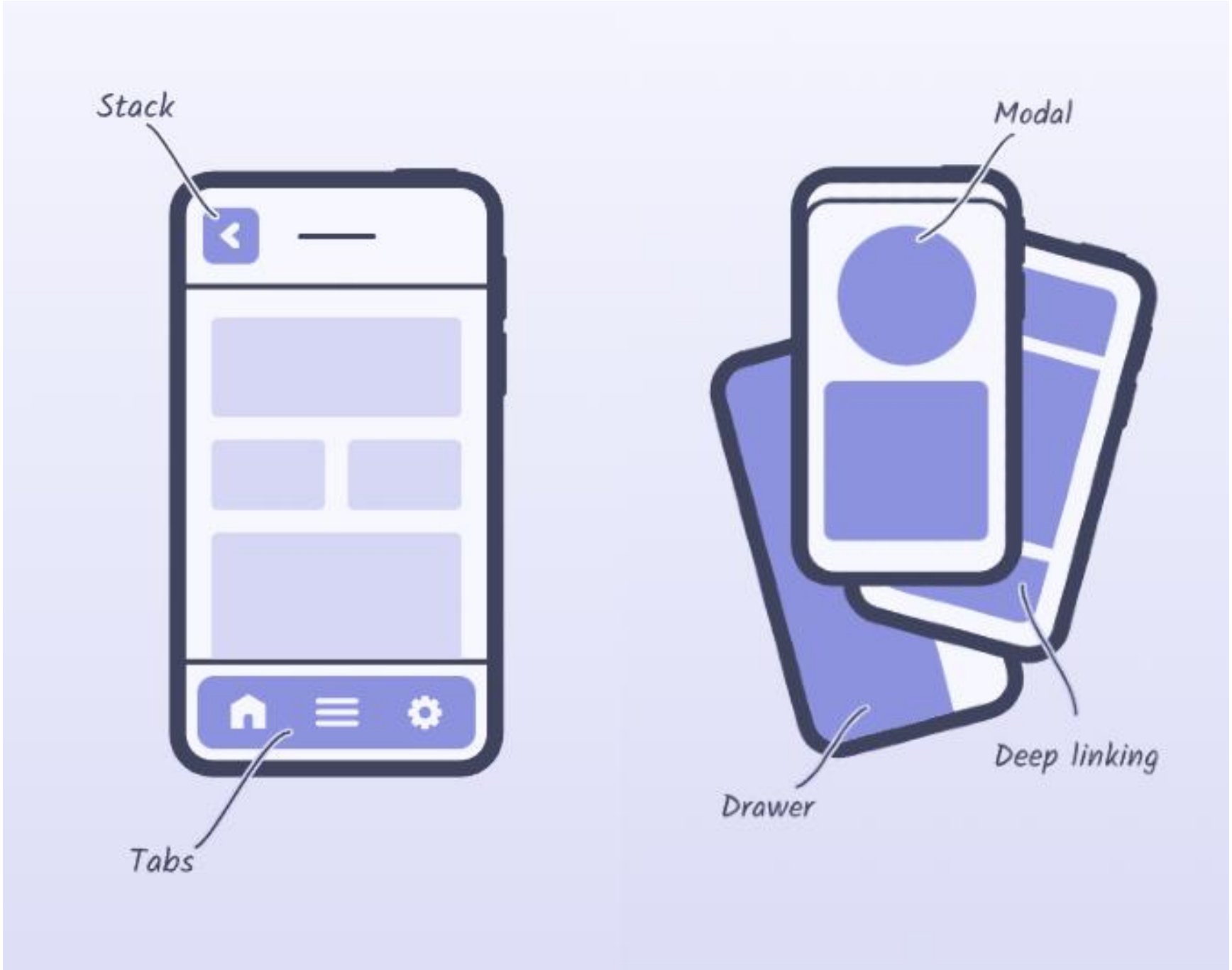
Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

# Aula 11 — Navegação: Stack Navigator

*Empilhando telas, passando dados entre elas e customizando o header*

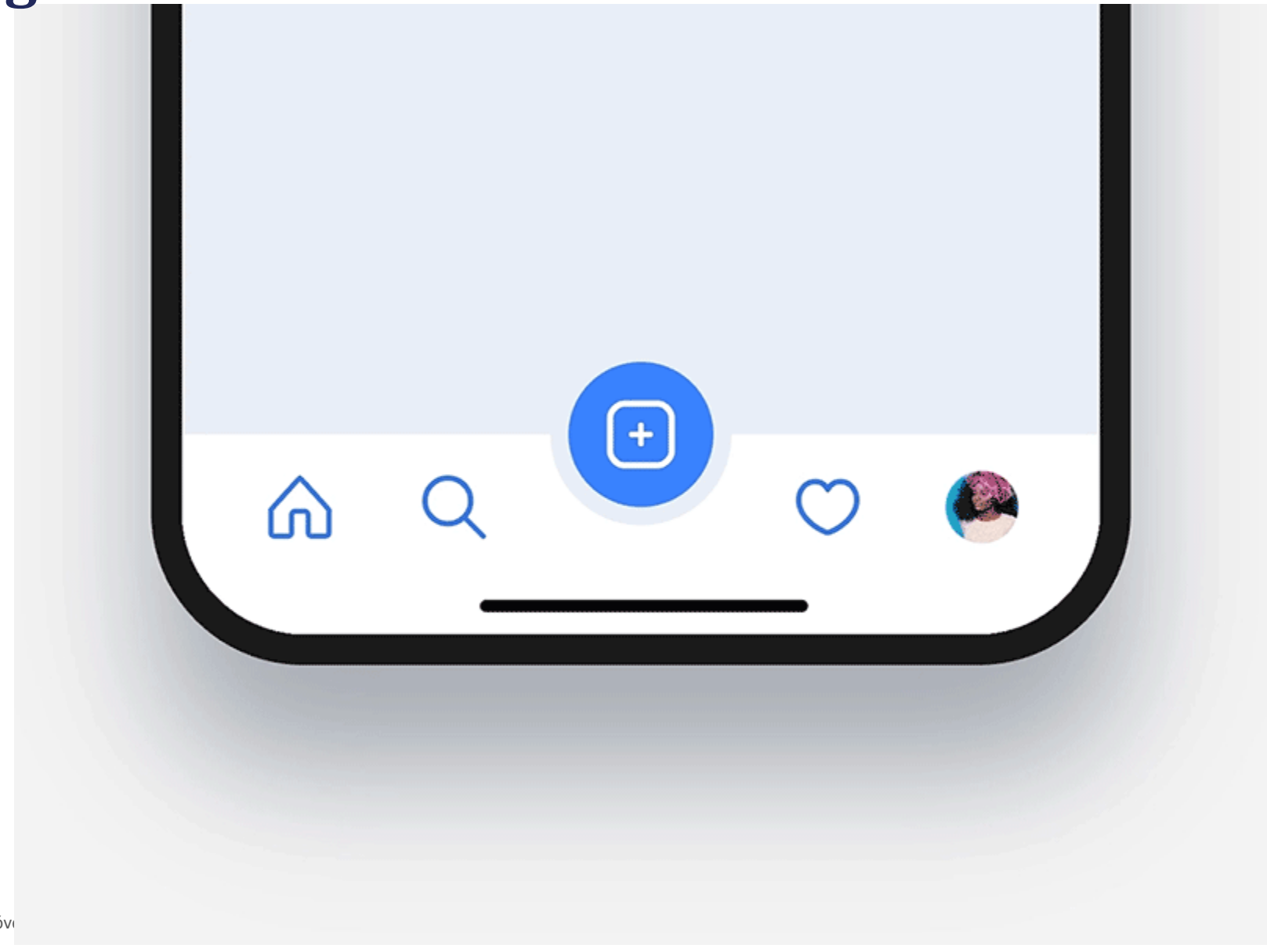
**Prof.: MSc. Alex Junior Nunes**
pfsalexnunes@gmail.com


Stack
Modal
Tabs
Drawer
Deep linking

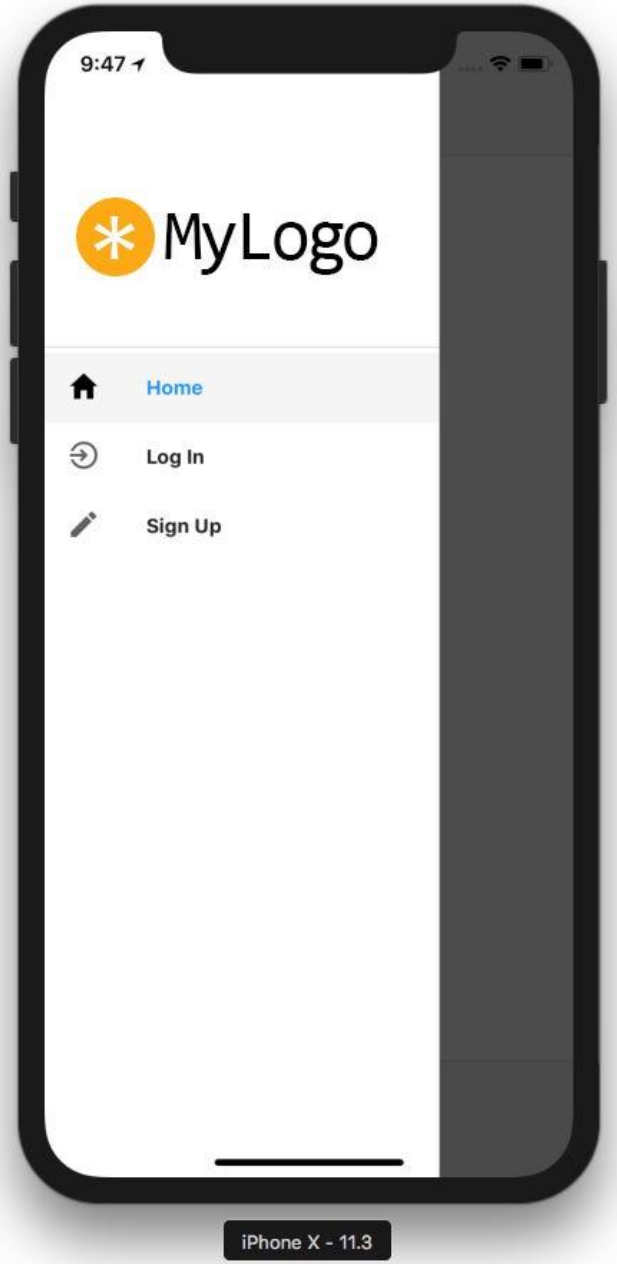
# Tab Navigator



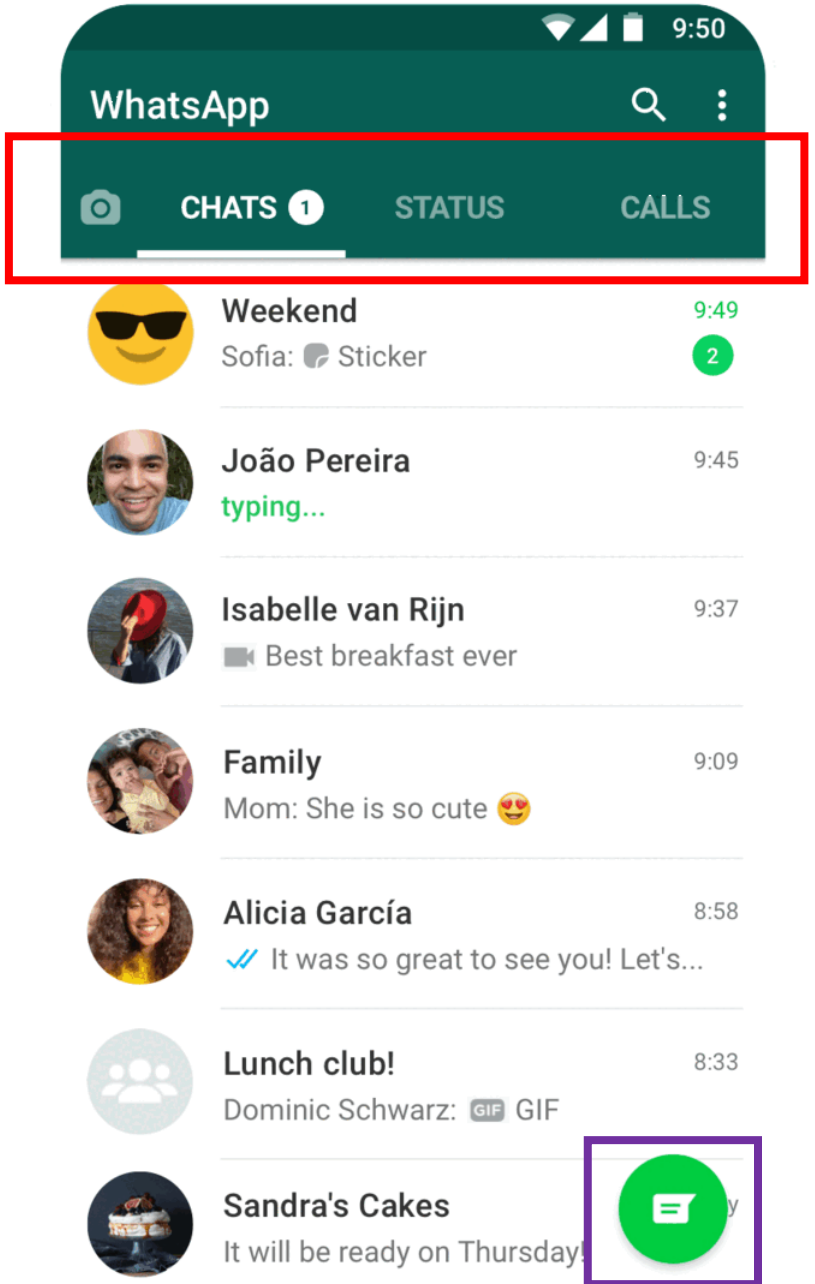
# Tab Navigator



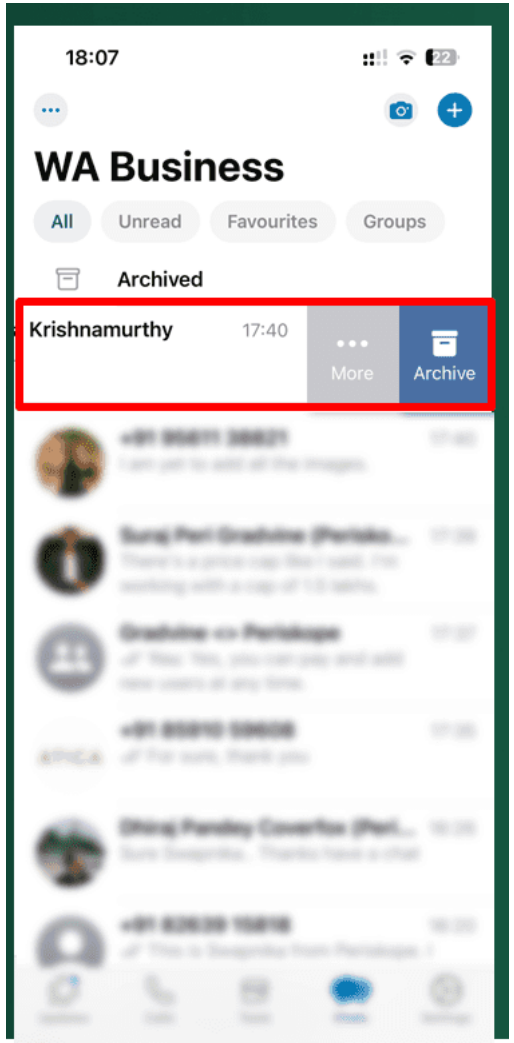
# Drawer

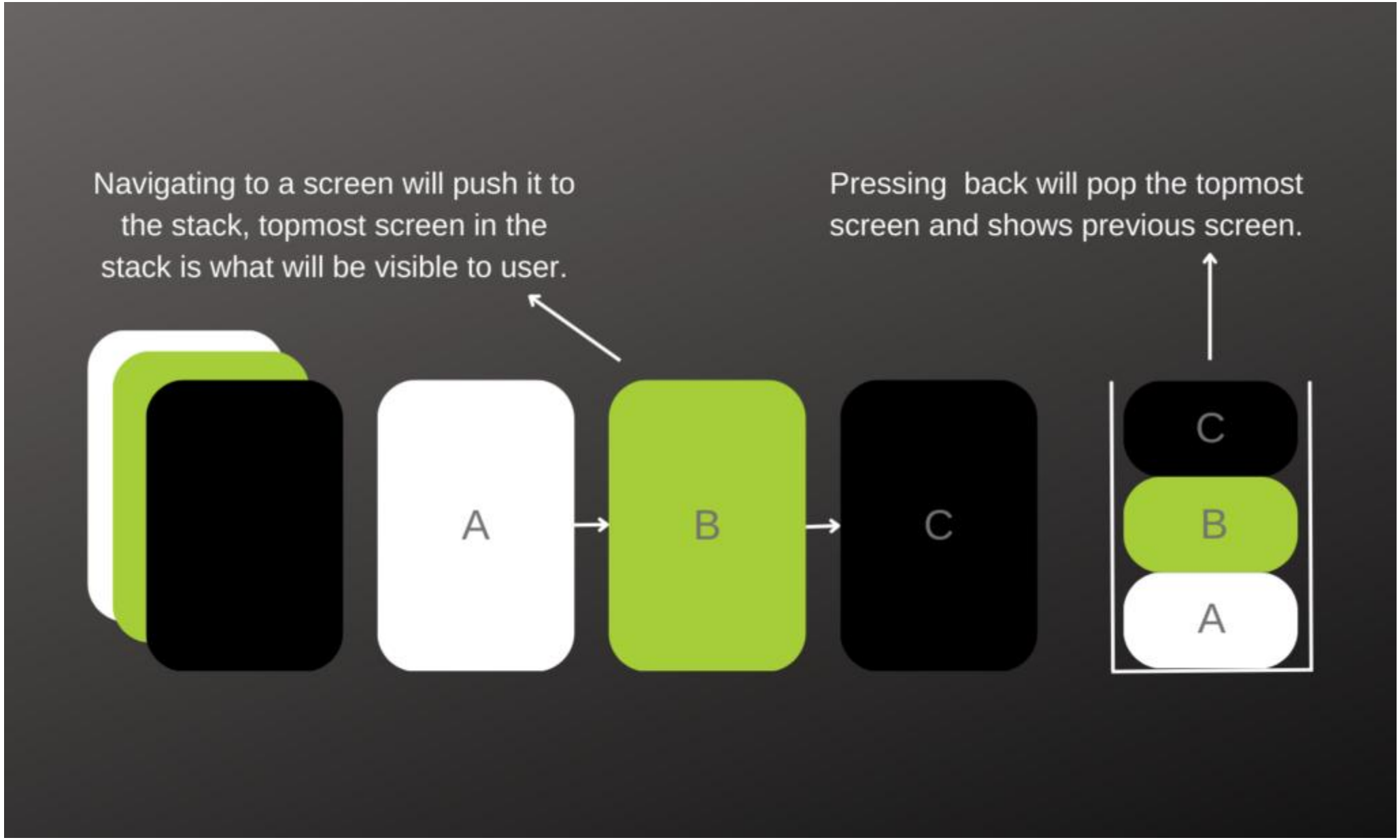


# Exemplos


9:50
WhatsApp
CHATS 1
STATUS
CALLS
Weekend
9:49
Sofia: Sticker
2
João Pereira
9:45
typing...
Isabelle van Rijn
9:37
Best breakfast ever
Family
9:09
Mom: She is so cute
Alicia García
8:58
It was so great to see you! Let's...
Lunch club!
8:33
Dominic Schwarz: GIF
Sandra's Cakes
It will be ready on Thursday!

# Exemplo


18:07
WA Business
All
Unread
Favourites
Groups
Archived
Krishnamurthy
17:40
More
Archive


Navigating to a screen will push it to the stack, topmost screen in the stack is what will be visible to user.
Pressing back will pop the topmost screen and shows previous screen.
A
B
C
C
B
A

CONCEITO

# Por Que Precisamos de um Navegador de Telas

Na Aula 06, trocamos de tela decidindo manualmente qual componente renderizar. Funciona, mas fica faltando um monte de coisa que qualquer app de verdade precisa.

### ANTES — troca manual (Aula 06)

- Sem botão de voltar automático
- Sem gesto de arrastar pra voltar (iOS) ou botão físico (Android)
- Sem histórico — só sabíamos a tela atual
- Header (se existisse) tinha que ser feito na mão

### AGORA — Stack Navigator

- Botão de voltar no header, automático
- Gesto e botão físico de voltar, de graça
- Histórico de telas — sabe de onde você veio
- Header pronto, com título e cores customizáveis

# Instalando e Configurando

```
npx expo install @react-navigation/native @react-navigation/native-stack react-native-screens react-native-safe-area-context
```

```
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      { /* todo o resto do app entra aqui */ }
    </NavigationContainer>
  );
}
```

*O NavigationContainer é único e envolve TODO o app — instale as bibliotecas uma vez só, por projeto.*

# Criando um Stack Navigator

```
import { createNativeStackNavigator }
  from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

<Stack.Navigator>
  <Stack.Screen
    name="Home"
    component={TelaHome} />
  <Stack.Screen
    name="Sobre"
    component={TelaSobre} />
</Stack.Navigator>
```

**name é o "apelido" da tela**

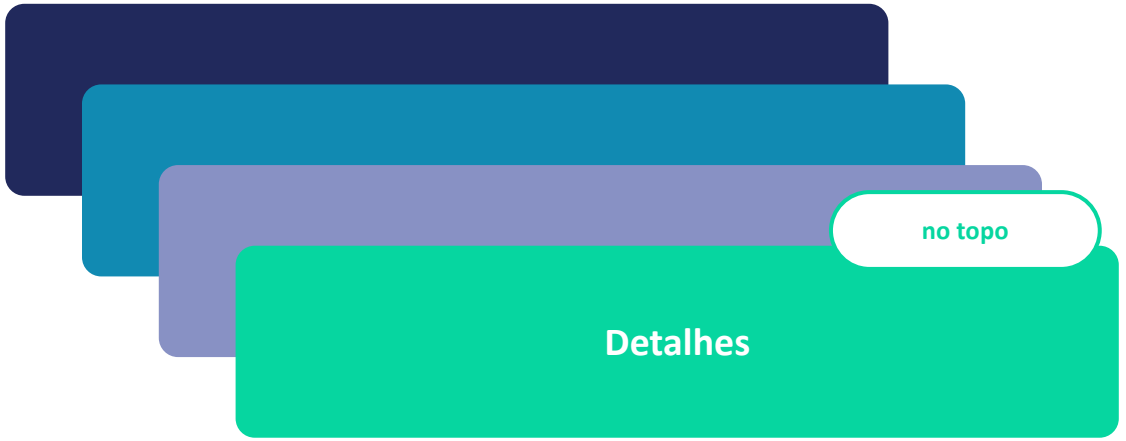
É esse texto que você usa depois em navigation.navigate('Sobre') — não precisa ser igual ao nome do arquivo.

**component é a tela de verdade**

O componente React que vai receber navigation e route como props, automaticamente.

*A ordem dos Stack.Screen não importa para a navegação — a PRIMEIRA registrada é sempre a tela inicial.*

# A Pilha: Empilhando e Desempilhando



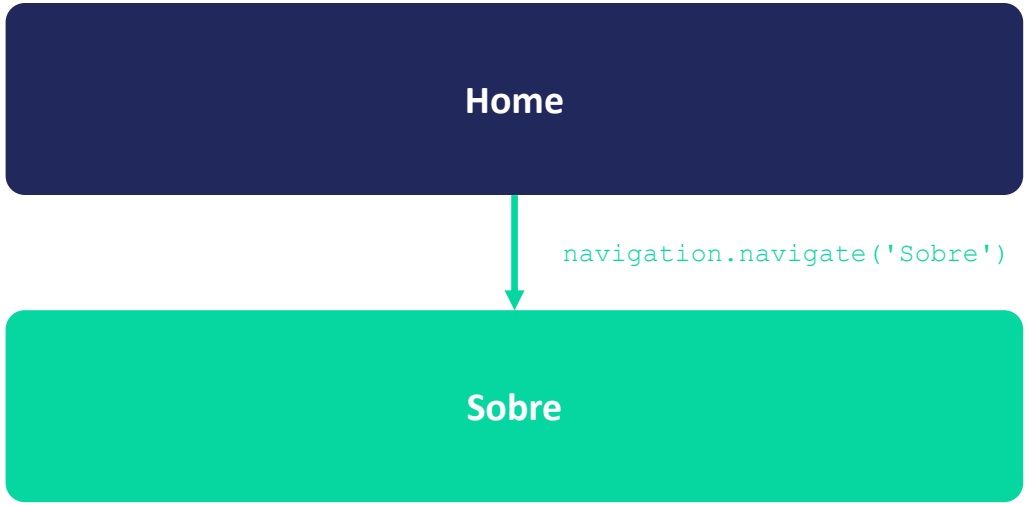

**navigate()**
empilha uma tela nova
no topo

**goBack()**
desempilha a tela
do topo

*É por isso que se chama STACK (pilha): só dá pra ver/tirar a tela do topo — nunca uma do meio.*

# Navegando Entre Telas

```
function TelaHome({ navigation }) {
  return (
    <Button
      title="Ir para Sobre"
      onPress={() =>
        navigation.navigate('Sobre')
      }
    />
  );
}
```




*`navigation` chega como prop em toda tela registrada no Stack.Navigator — automaticamente, sem Hooks. `navigate('Sobre')` usa exatamente o `name` que demos ao Stack.Screen.*

# Voltando para a Tela Anterior

```
<Button
  title="Voltar"
  onPress={() =>
    navigation.goBack()
  }
/>
```

*Voltar não é navegar pra uma tela nova — é desempilhar a atual e mostrar a que já estava embaixo dela, com o mesmo estado de antes.*

**Seta no header**

Aparece sozinha em toda tela que não é a primeira da pilha.

**Gesto (iOS) / botão físico (Android)**

Já funciona sem escrever nada de código.

**navigation.goBack()**

Faz a mesma coisa por código — útil pra um botão customizado.

**navigation.popToTop()**

Volta de uma vez até a primeira tela da pilha.

# Passando Dados Entre Telas

```
const aluno = { nome: 'Ana Souza', curso: 'ADS' };

navigation.navigate('Detalhes', aluno);
```

### 1º argumento

O nome da tela — o mesmo `name` do Stack.Screen.

### 2º argumento (opcional)

Um objeto qualquer com os dados que a próxima tela vai receber.

*Qualquer objeto serve: um aluno, um produto, um id — quem recebe é que decide o que fazer com ele.*

# Recebendo Dados na Tela de Destino

```
function TelaDetalhes({ route }) {

  const { nome, curso } =
    route.params;

  return (
    <Text>{nome}</Text>
  );
}
```

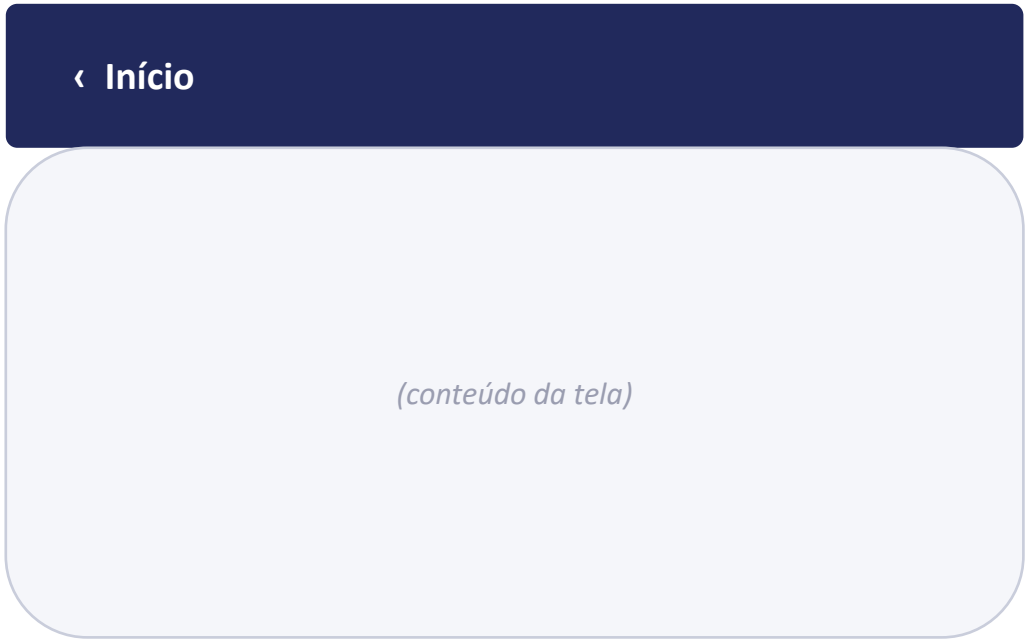
**Isso é a Aula 08 de novo!**

route.params é só um objeto comum — a mesma técnica de destructuring que já usamos pra ler props e pra ler objetos soltos serve aqui direto, sem nada novo pra aprender.

*Se a tela de destino não recebeu nenhum dado (route.params é undefined), tentar desestruturar direto quebra o app — vale sempre checar antes.*

# Customizando o Header

```
<Stack.Navigator
  screenOptions={{
    headerStyle: {
      backgroundColor: '#21295C' },
    headerTintColor: '#fff',
  }}
>
  <Stack.Screen ...
    options={{ title: 'Início' }} />
</Stack.Navigator>
```



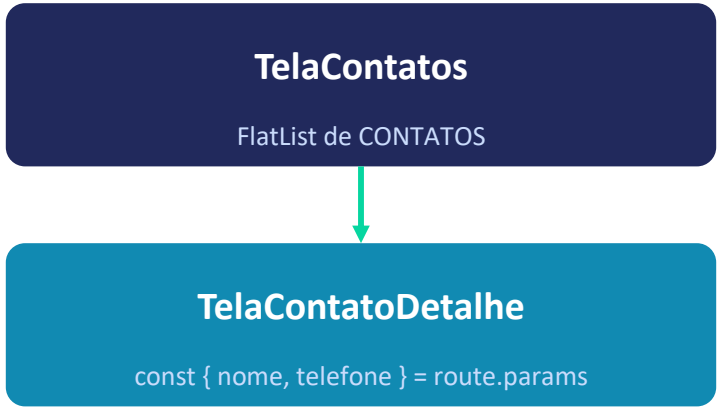

*screenOptions no Navigator = vale pra todas as telas.*
*options no Screen = vale só pra aquela tela (e tem prioridade).*

DESAFIO

# Desafio da Aula: Lista de Contatos

Em duplas, montem uma lista de contatos que navega para uma tela de detalhe:

- [ ] Array CONTATOS com pelo menos 3 objetos (id, nome, telefone)
- [ ] FlatList na TelaContatos usando esse array
- [ ] onPress de cada item navegando para ContatoDetalhe com o item
- [ ] TelaContatoDetalhe usando destructuring em route.params




```
chamada: navigate('ContatoDetalhe', item)
```

```
Pasta: 06-desafio-contatos/   ·   Gabarito: 06-desafio-contatos-solucao/
```

# O Que Levamos de Hoje

- NavigationContainer envolve o app; Stack.Navigator organiza as telas em pilha
- navigation e route chegam como props em toda tela do Stack — sem Hooks
- navigate(nome, dados) empilha uma tela nova; goBack() desempilha a atual
- route.params + destructuring é como a tela de destino lê os dados recebidos
- screenOptions (todas as telas) e options (uma tela) customizam o header

*Próxima aula: seguimos avançando no cronograma — conteúdo definido de acordo com o ritmo da turma.*